COMPTIA PENTEST+ · PT0-003 · DOMAIN 5.0

5.4 — Cleanup & Restoration

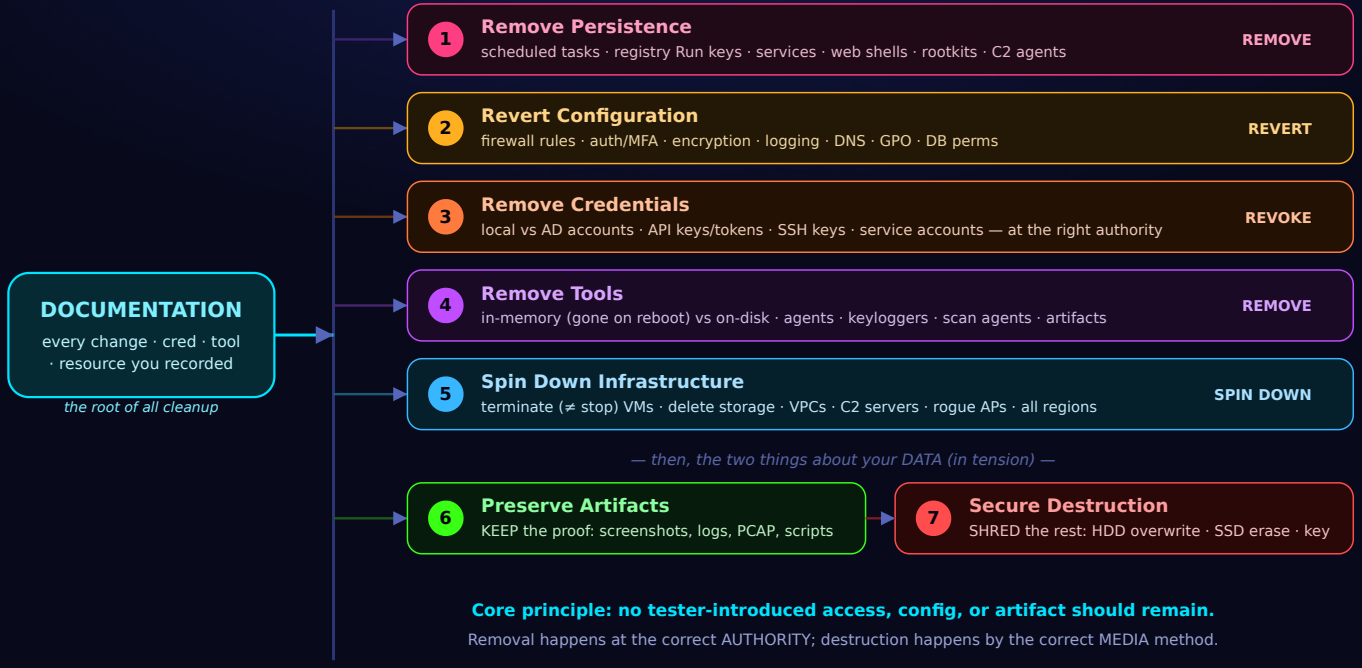
The final phase: return the environment to its **exact pre-test state**. Leave nothing you introduced behind. The exam treats this as both a technical discipline and a professional/ethical obligation — sloppy cleanup can leave a client *worse off* than before the test.

7 cleanup tasks | 1 documentation foundation | Preserve vs Destroy | "leave no trace"

The Cleanup Map — documentation is the root

Every task in this domain hangs off one thing: the **documentation you kept during testing**. You can't undo a change, revoke a credential, or delete a resource you didn't record. Five tasks *undo the mess*; the last two are a **Preserve-vs-Destroy** pair held in deliberate tension — keep what proves the work, shred what creates risk.

Remove · Revert · Spin down · Keep · Shred



Exam tell — the recurring theme across every task is "at the right level." Remove a credential at its **authority** (local box vs Domain Controller vs cloud IAM). Destroy data by its **medium** (HDD overwrite vs SSD secure-erase). Undo a change where it was **made**. Wrong level = the artifact survives.

MEMORY HOOK — THE 7 TASKS

"Pentesters Clean Carefully, Tools Included — Preserve, then Purge."

Persistence · **C**onfig · **C**redentials · **T**ools · **I**nfrastructure · **P**reserve · **P**urge (destroy).

1 Remove Persistence Mechanisms

REMOVE

You *deliberately* installed persistence to mimic a real attacker (survive reboots, resets, patching). By design it's hidden and resilient — so removal must be **systematic and complete**. Miss one and a real attacker inherits your backdoor.

THE PERSISTENCE CATALOG (KNOW EACH TYPE)

- **Scheduled tasks / cron** — Win Task Scheduler (`schtasks /delete`); Linux `/etc/crontab`, `/etc/cron.d/`, `crontab -e`. (e.g., a reverse shell that re-fires every 5 min.)
- **Services** — Win `sc create` / Linux systemd units in `/etc/systemd/system/`, often innocently named. Removal = stop → delete registration → remove files.
- **Registry Run keys (Windows)** — the classic. `HKLM\…\CurrentVersion\Run`, `HKCU\…\Run`, `HKLM\…\RunOnce`. Remove with `regedit` / `Remove-ItemProperty`.
- **Hidden files & rogue accounts** — dotfiles / hidden attrib; new admin accounts; direct edits to `/etc/passwd` & `/etc/shadow`. Remove at the **correct authority level** (local vs domain).
- **Backdoors** — web shells in web dirs, trojans, **rootkits** (kernel-level = hardest to detect/remove, may need specialized tools).
- **Browser extensions** — persist across sessions, steal creds; remove via the browser's extension manager.
- **C2 callbacks** — Empire / Covenant / Mythic agents beacon back. Removal = kill the agent process *and* delete the binary/script.

In-memory vs on-disk — in-memory persistence **disappears on reboot**; on-disk persistence **does not**. But even after a memory-resident agent is gone, the exploitation that placed it may have left logs / registry entries / connections that still need cleaning.

Picture it — you planted a dozen spare house keys — under the mat, in a fake rock, taped behind the gutter. Finding *one* and stopping isn't cleanup; a burglar just uses the next one. Layered persistence means you must walk the whole checklist, not eyeball it.

Exam tell — memorize the paths: `HKLM\…\Run / HKCU\…\Run` (Windows) and the Linux equivalents `/etc/init.d/, /etc/systemd/, crontab`. Expect "**what happens if persistence is NOT removed?**" → the org is left with exploitable re-entry paths. And accounts come out at the **right authority** (local vs AD domain controller).

HKLM\Run · HKCU\Run · schtasks · crontab -e · sc delete · web shell · rootkit · C2 / beacon · domain controller

2 Revert Configuration Changes

REVERT

You bent settings to make exploitation possible; now bend them back **precisely**. Miss one and you've *introduced* a vulnerability — open port, disabled logging, weakened auth — the exact opposite of the engagement's purpose.

WHAT GETS CHANGED (AND MUST BE RESTORED)

- **Network / firewall** — temp rules for reverse shells & C2, NAT/port-forwarding, routing, ACLs → back to restrictive defaults.
- **Security settings** — re-enable MFA, restore password complexity, re-harden encryption (drop SSLv3 / weak ciphers), re-enable UAC, restore AppLocker/whitelisting.
- **Logging & monitoring** — re-enable log forwarding, restore SIEM rules/thresholds, revert Event Log settings. Leaving logging off = a dangerous blind spot for defenders.
- **Database** — remove test accounts, revoke escalation perms, drop stored procs/triggers added during SQLi testing.
- **DNS** — restore records changed for on-path/harvesting, revert split-horizon and TTL tweaks.
- **Access controls** — revert modified **GPOs**, file/dir permissions, share permissions, and re-strengthen weakened password policies.

THE ENABLER: DOCUMENTATION

Accurate reversion depends entirely on records kept *during* testing: system · setting · **original value** · test value · timestamp · reason. **Snapshots / config backups** give an easy rollback path for complex systems.

> **Picture it —** a stagehand's spike marks. Before you move the furniture you tape the floor where each piece stood — so at the end of the night everything goes back on its exact mark. No tape (documentation) = you're guessing.

> **Exam tell —** scenario type: "which change left a **residual vulnerability**?" → look for the setting that *weakens* security and wasn't restored (open firewall port, disabled logging, MFA off, weak cipher enabled). Keywords: snapshot/rollback, GPO, ACL restoration, logging re-enabled.

firewall rule reversion · snapshot / rollback · logging re-enabled · GPO · ACL restoration · DNS record · encryption settings

3 Remove Tester-Created Credentials

REVOKE @ SOURCE

Accounts, API keys, service accounts, tokens created during testing are **live authentication pathways**. Left behind, they're permanent backdoors — often over-privileged, weak-passworded, outside rotation, and unknown to the security team.

CREDENTIAL TYPES & WHERE THEY DIE

- **Local accounts** (Win SAM / Linux `/etc/passwd` + `shadow`) — `net user [name] /delete`, `userdel`. Deleting on one box does *not* touch others; each is separate.
- **Active Directory accounts** — domain-wide, usable from any joined machine. **Cannot be removed from a local workstation** — delete at the **Domain Controller** (ADUC or `Remove-ADUser`). ← heavily tested.
- **API keys / tokens** — must be **explicitly revoked in the issuing service** (cloud IAM, app console). Deleting a *script that references* a key does **NOT** invalidate the key. Same for OAuth/JWT/session tokens.
- **Service accounts** — often specially privileged; check both local SAM *and* AD.
- **SSH keys & certs** — remove test entries from `~/.ssh/authorized_keys`; pull test certs from cert stores.

Authentication-authority awareness (the #1 idea) — remove at the authoritative source: local → locally · domain → Domain Controller · cloud → cloud IAM · API key → issuing service. Tightly-integrated systems (wikis, audit platforms) may need **direct database removal** to preserve audit integrity.

Picture it — you deleted the sticky note with the office door code — but the *keypad* still accepts that code. Shredding the note (the script) ≠ changing the lock (revoking the key). You have to reprogram the lock at the door.

Exam tell — the two trap questions: ① "AD account still works from another machine" → wrong authority, remove at **Domain Controller**. ② "deleted the script but the key still works" → **revoke the API key at the issuing service**. Any "where do you remove it?" → match credential type to authority.

domain controller · net user /delete · userdel · API key revocation · authorized_keys · authentication authority · service account

4 Remove Tools

REMOVE

Frameworks, keyloggers, scanners, proxy agents, custom scripts — legit during the test, a liability after. “Just delete the files” is harder than it sounds.

WHY IT’S NON-TRIVIAL

- Tools inject into processes, modify binaries, spread across dirs.
- **Simple deletion is recoverable** (forensics) until space is overwritten → sensitive artifacts need **secure deletion** (§7).
- Secondary artifacts linger — logs, config, temp files, registry entries.
- **Automation may re-deploy** — a scheduled task can reinstall a “deleted” tool.

CATEGORIES TO CLEAR

- **In-memory payloads** (Meterpreter/Empire in memory) — gone on reboot, but **verify**, don’t assume; the placing exploit may have left other artifacts.
- **Persistent executables / scripts** — .exe/.dll/.so, PS/Python/Bash/batch, web shells (PHP/ASPX/JSP) in web roots.
- **C2 agents** — Empire stagers, Covenant implants, Mythic agents; may hide as DLLs, registry-encoded payloads, or **WMI subscriptions**. (Infra listeners → §5.)
- **Keyloggers / monitoring** — remove the binary *and* captured-data files.
- **Vuln-scan agents** — use the scanner’s **official uninstaller**, not a plain delete.
- **Supporting files** — Metasploit resource files, Nmap outputs, temp dirs, tool logs on target.

Tools ↔ Persistence are linked — if a scheduled task/cron re-runs a tool, deleting the binary alone leaves the task to error out or drop another payload. Always cross-reference tool removal with §1 persistence removal.

Picture it — pulling a weed by the leaves vs the root. Delete the visible binary and the automation (root) grows it right back. And a “deleted” file in the trash is a weed you dropped on the lawn — still there until you actually burn it (secure delete).

Exam tell — “tester removed the tool but it **came back**” → the answer removes the **automation task too**. “in-memory vs on-disk” → memory clears on reboot (verify), disk needs deliberate + often secure deletion. Scan agents → **proper uninstall**.

in-memory payload | persistent executable | secure deletion | web shell | C2 agent | WMI subscription | automation task | artifact

5 Spin Down Infrastructure

SPIN DOWN

Dedicated testing infra — VMs, cloud instances, C2 servers, rogue APs, networks — must be **intentionally terminated**. Leaving it up = security, financial, legal, and operational risk.

WHAT TO DECOMMISSION

- **VMs** — local hypervisor VMs deleted (not just powered off). Cloud instances **terminated, not stopped**. Delete snapshots & AMIs from test VMs too.
- **Storage** — EBS/Managed Disks, S3/Blob/Cloud Storage buckets emptied & deleted; backups/snapshots removed. **Storage persists independently of compute** — separate, explicit deletion.
- **Networks** — VPCs/virtual networks, VPNs, peering to prod, custom route tables.
- **Security groups / firewall rules** — delete testing rules; don't leave overly-permissive convenience rules.
- **C2 infra** — shut down listeners/team servers (Empire/Covenant/Cobalt Strike); let phishing/redirector domains expire or deregister; **revoke test TLS certs**.
- **Rogue APs** — evil-twin/rogue APs must be **physically removed**; stop their DHCP to end disruption.
- **Log/monitoring infra** — shut down test SIEM/log aggregators; handle their logs per SOW.

Stop ≠ Terminate — a **stopped** cloud instance still accrues storage cost and can be restarted. Only **terminate** permanently deletes it. This distinction is a guaranteed exam point.

The multi-region trap — resources can be scattered across regions (us-east-1, eu-west-1, ap-southeast-1). Consoles don't show a global view by default → **review every region**. Backups & snapshots are the most-overlooked leftovers.

Picture it — striking a film set. It's not enough to turn off the lights (stop) — you tear down the rigging, return the rentals, and clear *every* location you shot in (every region), or you keep paying rent on a warehouse you forgot about.

Exam tell — "terminate (not stop)," "storage volume persists," "multiple regions," "rogue AP physically removed," "snapshot/AMI." "What risk remains if C2 is left active?" → attacker can discover & leverage it.

terminate vs stop | S3 bucket | storage volume | multi-region | C2 / team server | rogue access point | VPC | snapshot / AMI

6 Preserve Artifacts

KEEP · THE EXCEPTION

The one task that's *retention*, not removal. Artifacts are your **proof of work** — what was tested, how access was gained, what was exploited. They enable defensible reporting, reproducibility/retesting, legal defensibility, and future comparison.

WHAT TO PRESERVE

- **Screenshots** — shell access, DB contents, privileged commands; label with system, IP, timestamp, and the finding.
- **Logs & output** — Nmap/Nessus/Burp output, terminal session logs, target event logs.
- **Custom scripts/code** — your exploits & automation, commented, for reproducibility.
- **Communication records** — client/team emails, chat, notes, **authorization letters**, scope-change approvals.
- **Network captures (PCAP)** — irrefutable evidence of traffic, credential captures, lateral movement.
- **Config & vuln data** — insecure config copies (redacted), scan reports, documented exploit code.

STORE IT SECURELY

- **Access controls** (RBAC — authorized team only) · **encryption at rest** · **backups** (loss is irreversible) · **retention period defined in the SOW**, then securely destroyed (§7).

Real sensitive data inside artifacts — if you captured real creds/PII/financial/health data: **minimize** (keep only what documents the finding), **protect** (highest controls), **anonymize/redact** where non-essential, and handle per the SOW.

Picture it — a crime-scene evidence locker. You don't wipe the scene clean and toss everything — the photos, prints, and chain-of-custody logs get sealed, labeled, and locked away, because they're what stands up later.

Exam tell — this is the **exception to "remove everything."** "Which items do you **keep**?" → screenshots, logs, PCAP, scripts, comms. "What about real sensitive data found?" → **minimize & protect**. Retention length → **the SOW**.

screenshot · PCAP / network capture · custom script · encryption at rest · access controls · retention period (SOW) · data minimization

7 Secure Data Destruction

SHRED · BY MEDIUM

When data is no longer needed (or the SOW retention expires), destroy it **irrecoverably**. Simple delete only removes the directory entry — the bytes stay on the medium until overwritten, and **file carving** recovers them. Unacceptable for creds, vuln data, and network maps.

DESTRUCTION TECHNIQUES

- **Shredding / overwriting** — overwrite the space (zeros / random / multiple **passes**) before removing the entry. Linux `shred -vzn 3 file`; Windows `sdelete` (Sysinternals). More passes = more thorough.
- **SSD secure erase** — **overwriting does NOT work reliably on SSDs** because of **wear leveling** (the controller writes new data to different physical cells, leaving originals intact). Use manufacturer **ATA Secure Erase / NVMe Sanitize**. OS-level shredders are unreliable here.
- **Encryption + key destruction** — if encrypted from the start, just **destroy the key** → data is cryptographically unrecoverable. (Why encrypting test data early is best practice — it makes destruction trivial.)
- **Physical destruction** — degaussing (**HDD only**, not SSD/optical), shredding the drive, incineration. Reserved for end-of-life hardware, not routine data.

STORAGE TYPE	RECOMMENDED METHOD
HDD (spinning)	`shred`, `sdelete`, multi-pass overwrite
SSD	Manufacturer secure erase (**ATA Secure Erase**, NVMe Sanitize)
Encrypted storage	Destroy the **encryption key**
Cloud storage	Provider secure-delete / bucket deletion
Physical (end-of-life)	Degauss (HDD only), shred, incinerate

Picture it — deleting a file is crossing a name out of the phone book — the house is still there. Shredding bulldozes the house. And an SSD is a house that *teleports* when you try to bulldoze it (wear leveling) — you need the builder's demolition order (secure erase), not your own wrecking ball.

Exam tell — the money concepts: simple deletion is insufficient (file carving); HDD → overwrite/shred but SSD → secure erase because of wear leveling; encrypted → destroy the key; degaussing = HDD only. Know `shred` as the Linux tool and "passes" = overwrite cycles.

file carving · shred · sdelete · wear leveling · ATA Secure Erase · passes · key destruction · degaussing (HDD only)

Mnemonic Vault

PCCTI · PP

“Pentesters Clean Carefully, Tools Included — Preserve, then Purge.” The 7 tasks: Persistence · Config · Credentials · Tools · Infrastructure · Preserve · Purge.

Terminate ≠ Stop

Stopped = still billing, can restart. **Terminate** = permanently gone. Applies to every cloud instance.

Shred HDD · Erase SSD

Wear leveling breaks overwriting on SSDs → use ATA Secure Erase / NVMe Sanitize. HDDs overwrite fine.

Revoke at the Source

Kill creds at their authority: local→local · domain→**DC** · cloud→IAM · **API key→issuing service** (deleting the script isn’t enough).

Preserve then Purge

§6 vs §7 in tension: **keep** the proof (screenshots, logs, PCAP, scripts), **shred** the risk (creds, vuln data). SOW sets retention.

Root = Docs

You can only undo what you **documented**: system · setting · original value · timestamp · reason. Snapshots = safety net.

⚔ Confusables Cram — the pairs the exam blurs

IF YOU SEE…	RIGHT ANSWER	NOT THIS — BECAUSE
Cloud instance “still incurs cost / can be restarted”	**Terminate** it	Not **Stop** — stopping pauses but keeps storage & the ability to restart
“Deleted” files recovered by forensics	**Secure deletion / shred**	Not simple delete — that leaves the bytes (file carving recovers them)
Overwrote an **SSD** but data survived	**ATA Secure Erase** (wear leveling)	Not multi-pass overwrite — the controller relocates writes, sparing originals
AD account still works from another machine	Remove at the **Domain Controller**	Not local delete — the account is domain-wide, not on the workstation
Deleted the script but the API key still authenticates	**Revoke the key** in the issuing service	Not deleting the script — the key lives in the platform, not the file
Tool reappears after you deleted it	Remove the **automation task** too (§1)	Not just the binary — the scheduled task/cron re-deploys it
Encrypted disk, need fast destruction	**Destroy the encryption key**	Not full overwrite — no key = data is cryptographically dead
Which items do you **KEEP**?	**Preserve artifacts** (proof of work)	Not destroy — screenshots/logs/PCAP/scripts support the report & retesting
Storage still there after the VM is gone	Delete **storage volumes/buckets** separately	Not covered by termination — storage persists independently of compute
Magnetic erasure of a drive	**Degaussing** — HDD only	Not for SSD/optical — no magnetic domains to disrupt
In-memory agent vs on-disk backdoor	Memory clears on **reboot** (verify); disk needs deliberate deletion	Don’t assume reboot cleans disk persistence — it doesn’t

⌘ Scenario Decoder — stem phrase → action

SCENARIO IN THE STEM	CORRECT ACTION
Reverse shell keeps re-firing on a schedule	Remove the scheduled task / cron (persistence)
Program launches at every login on Windows	Delete the **HKLM\Run / HKCU\Run** value
Firewall port / logging still off after test	**Revert configuration** — restore rule, re-enable logging
Test account works from any domain machine	Delete at the **Domain Controller** (ADUC / Remove-ADUser)
Script deleted, but API key still valid	**Revoke** the key in the cloud IAM / issuing service
Removed the tool, it reinstalled itself	Remove the linked **automation task** as well
Cloud VM paused but still billing / restartable	**Terminate** the instance (not stop)
Resources found in another region	Review **every region** for leftovers
"Deleted" data recovered from an HDD	**Shred / overwrite** (multi-pass)
Overwrite failed to erase an SSD	**ATA Secure Erase / NVMe Sanitize** (wear leveling)
Encrypted media, destroy quickly & certainly	Destroy the **encryption key**
Which evidence do we retain, and how?	**Preserve** — encrypt, access-control, per **SOW** retention
Real PII captured during testing	**Minimize, protect, anonymize**; handle per SOW

THE 5 EXAM CORRELATIONS (MULTI-CONCEPT SCENARIOS)

- **Persistence ↔ Tools** — a task that re-deploys a tool: remove *both*.
- **Credentials ↔ Infrastructure** — shutting a C2 server doesn't revoke the keys that managed it; those keys can hit *other* resources.
- **Config ↔ Logging** — reverting config includes *re-enabling logging*, or defenders go blind.
- **Artifacts ↔ Destruction** — preserve what supports the report; destroy what's no longer needed.
- **Documentation ↔ Everything** — no records → no complete cleanup.

★ Night-Before Cheat Strip

CORE PRINCIPLE

No tester-introduced access, config, or artifact remains. Sloppy cleanup can leave the client **worse off** — that’s the ethical hook.

THE 7 TASKS

Persistence · **C**onfig · **C**redentials · **T**ools · **I**nfrastructure · **P**reserve · **P**urge. All rooted in **documentation**.

TERMINATE ≠ STOP

Stop = still billing + restartable. **Terminate** = permanent. Storage & snapshots persist separately — delete them too.

HDD VS SSD

HDD → `shred`/overwrite. SSD → **ATA Secure Erase** (wear leveling breaks overwriting). Encrypted → **destroy the key**.

REMOVE AT THE AUTHORITY

Local→local · **AD→Domain Controller** · cloud→IAM · **API key→revoke in issuing service**. Deleting a script ≠ revoking the key.

PERSISTENCE PATHS

HKLM\…\Run, **HKCU\…\Run**, RunOnce · schtasks · cron · services · web shells · rootkits (kernel, hardest) · C2 beacons.

CONFIG: DON’T FORGET LOGGING

Re-enable logging/SIEM, restore MFA/ciphers/GPO/ACLs/DNS. Disabled logging = a blind spot for defenders.

TOOLS ↔ AUTOMATION

Tool came back? Kill the **scheduled task** that re-deploys it. In-memory clears on reboot (verify); on-disk doesn’t.

PRESERVE (THE EXCEPTION)

Keep screenshots, logs, **PCAP**, scripts, comms. Encrypt at rest + RBAC. Retention set by the **SOW**.

SIMPLE DELETE FAILS

File carving recovers "deleted" files. Sensitive artifacts need secure deletion. Multi-**pass** = more thorough.

Domain 5.4 · Cleanup & Restoration — the closing phase of the engagement lifecycle.
Study aid for CompTIA PenTest+ PT0-003. Cleanup is a professional and ethical obligation on authorized, in-scope engagements only.